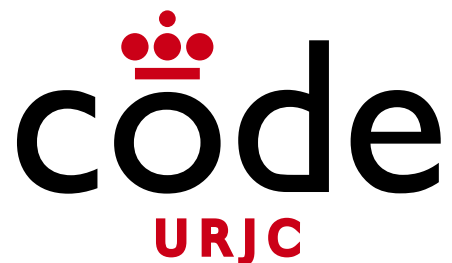


Fundamentos de la Web

Bloque III: Tecnologías de servidor web

Tema 3.1: Introducción a JavaScript



©2024

Micael Gallego, Francisco Gortázar, Michel Maes, Óscar Soto, Iván Chicano

Algunos derechos reservados

Este documento se distribuye bajo la licencia
“Atribución-CompartirIgual 4.0 Internacional”
de Creative Commons Disponible en
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Introducción a JavaScript

- JavaScript
- JavaScript en el navegador
- JavaScript fuera del navegador



Introducción a JavaScript

- JavaScript
- JavaScript en el navegador
- JavaScript fuera del navegador



JavaScript

- Es un lenguaje de programación basado en el estándar **ECMAScript** de **ECMA** (otra organización diferente al **W3C**)
- Hay ligeras **diferencias** en la implementación de JS de los navegadores y runtimes, aunque actualmente todos son bastante **compatibles** entre sí (**en el pasado no fue así**)

<https://ecma-international.org/>



- JavaScript no es Java
 - Aunque algunos elementos de la sintaxis recuerden a **Java**, son lenguajes **completamente diferentes**
 - El nombre **JavaScript** se eligió al publicar el lenguaje en una época en la que **Java estaba en auge** y fue principalmente por marketing (inicialmente se llamó **LiveScript**)

- **Características principales de JavaScript**
 - **Scripting:** No necesita compilador. Se ejecuta directamente desde el código fuente usando un *Runtime*
 - **Tipado dinámico:** Habitual en los lenguajes de script
 - **Funcional:** Las funciones son elementos de primer orden (se puede asignar a variables, pasar como parámetro...)
 - **Orientado a objetos:** De forma más dinámica que Java

Referencias

- **Documentación**

- <https://developer.mozilla.org/es/docs/Web/JavaScript>

- **Tutoriales interactivos**

- <https://www.codecademy.com/>
 - <https://www.pluralsight.com/courses/javascript-getting-started>

Introducción a JavaScript

- JavaScript
- **JavaScript en el navegador**
- JavaScript fuera del navegador



JavaScript

- Las **páginas web** pueden incorporar **interactividad** con el lenguaje **JavaScript**
- Con **JavaScript** se puede **modificar la página** (manipulando modelo de objetos del documento **DOM**)
- También se pueden hacer **peticiones al servidor web** en segundo plano y **actualizar el contenido de la web** con los resultados (AJAX)

Ejercicio 1

Hello world!

ejer1-hello-world

- Crear una carpeta
- Crear un fichero **app.js**

```
console.log("Hello World!");
```

Ejercicio 1

Hello world!

ejer1-hello-world

- Crear un fichero **index.html**

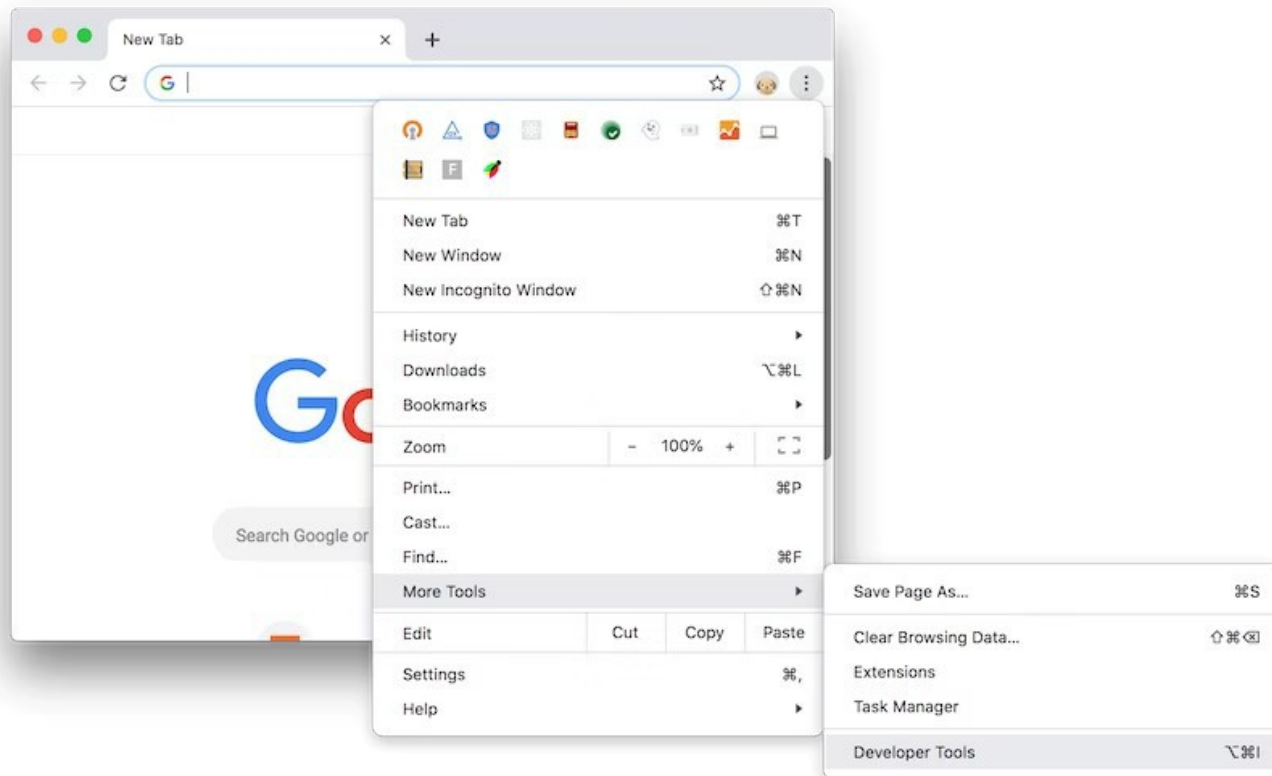
```
<html>
  <head>
    <script src="app.js"></script>
  </head>
  <body></body>
</html>
```

Ejecuta
el código JS al
cargar la página
en el navegador

- Ábrelo en el navegador y muestra la consola

Ejercicio 1

- **Mostrar información desde JavaScript**
- **Mostrar la consola de Google Chrome**



Ejercicio 1

- **Mostrar información desde JavaScript**
 - Escribiendo en la consola JavaScript del navegador

```
console.log("Text");
```

- Más adelante veremos cómo mostrar información en la página web

Introducción a JavaScript

- JavaScript
- JavaScript en el navegador
- **JavaScript fuera del navegador**



JavaScript fuera del navegador

- Existen varios runtimes para ejecutar JavaScript fuera del navegador
 - **Node.js** (el más popular y usado)
 - Bump
 - Deno



JavaScript fuera del navegador

¿Qué es Node.js?

- Es un *runtime* que permite ejecutar **JavaScript** fuera del navegador (existen otros)
- Usa el mismo motor JS que Google Chrome (V8)
- Se usa para implementar:
 - **Parte servidor de aplicaciones web**
 - Herramientas de desarrollo web
 - Entornos de desarrollo (VSCode)



<https://nodejs.org/en>

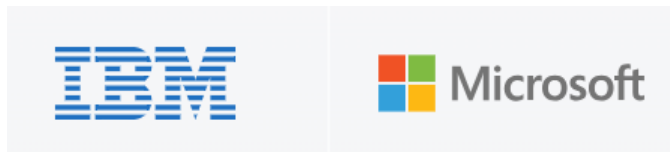
Soporte en la industria



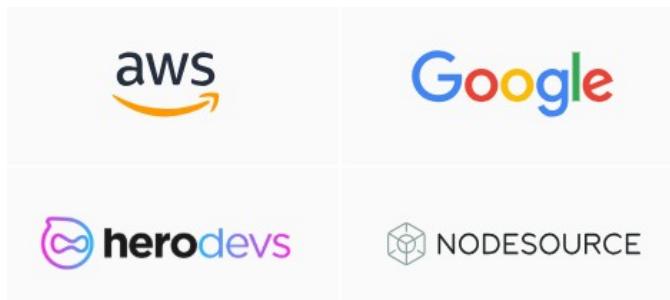
Silver

Platinum



Gold



Plataforma con mucha adopción

- Tiene una **gran comunidad de desarrolladores**
- Está soportado de forma oficial en los **proveedores cloud** (como Java, Python...)
- Tiene una **gran cantidad de librerías**
- Cada vez se usa más a nivel **empresarial**
- Incorpora las últimas versiones del estándar **EcmaScript** (lenguaje y librerías muy completas)

Tipos de aplicaciones

- Con Node.js se pueden desarrollar todo tipo de aplicaciones:
 - Servicios web
 - Herramientas por línea de comandos (CLI)
 - Aplicaciones web SPA
 - Aplicaciones con interfaz gráfico de usuario

Servicios web

- Aplicaciones web MVC: Generación de HTML en servidor
- Backend para aplicaciones SPA
- Comunicación: API REST, WebSockets, AMQP, gRPC...
- Acceso a base de datos: Relacionales, NoSQL...
- Serverless, Microservicios...



Herramientas por línea de comandos

- Herramientas para desarrollo:
 - Linters
 - Compiladores
 - Testing de rendimiento
- Gestión de proyectos: angular-cli, vue-cli...
- Scripts de sistemas



Node.js

Aplicaciones con interfaz gráfico de usuario

- Se combina una aplicación **Node.js** (para acceso al sistema) con un navegador web (**Chromium**) para la interfaz de usuario



<https://electronjs.org>



<https://nwjs.io>

Librerías

- En **Node.js** no se pueden usar librerías del browser como **DOM**, **BOM**, etc. porque no hay interfaz de usuario
- Node.js ofrece por defecto unas mínimas librerías para interactuar con el sistema operativo
- Existen muchas librerías libres disponibles en NPM, el gestor de paquetes oficial de Node



<https://npmjs.com>

Librerías

- Las librerías incluidas por defecto en Node.js ofrecen las siguientes funcionalidades:
 - Gestión de procesos
 - Gestión de datos en memoria nativa del sistema
 - Acceso a la línea de comandos
 - Sistema de ficheros
 - Redes (Http, Sockets)
 - Flujos de bytes

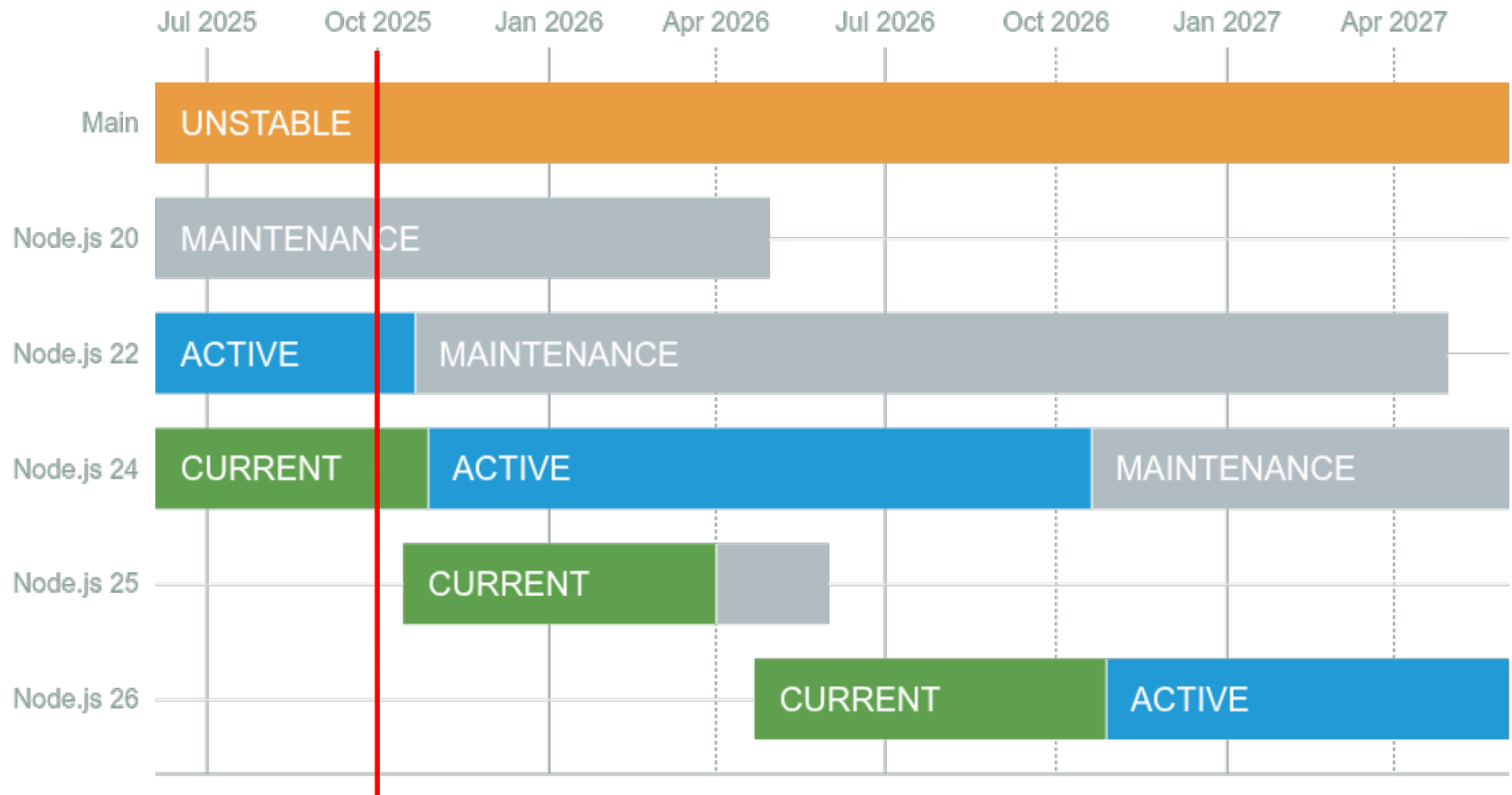
<https://nodejs.org/api>

Versiónes

- Node.js ofrece 2 tipos de versiones
 - Activa LTS (*Long Term Support*): Soporte extendido
 - Actual (*Current*): últimas funcionalidades
- En Junio de 2024 las versiones son:
 - Activa LTS: **22.20.0**
 - Actual: **24.10.0**

<https://github.com/nodejs/LTS#lts-schedule1>

Versiones



Rendimiento

- El código **JavaScript** no es tan eficiente como Java o C# debido a su tipado dinámico
- Arranca muy rápido, lo que se valora en arquitecturas **serverless** (que arrancan procesos por cada petición)
- Su modelo de programación **no bloqueante (asíncrono)**, le ha permitido crear soluciones más escalables que usando el **modelo bloqueante** (que era el más usado en otras plataformas)
- El **modelo de programación es mucho más sencillo** que otras alternativas al no existir varios hilos de ejecución

Instalación

Run JavaScript Everywhere

Node.js® is a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line tools and scripts.

Get Node.js®

Get security support
for EOL Node.js versions



Node.js is proudly supported by the partners above and more.

Create an HTTP Server Write Tests Read and Hash a File Streams Pipeline Work with Threads

```
1 // server.mjs
2 import { createServer } from 'node:http';
3
4 const server = createServer((req, res) => {
5   res.writeHead(200, { 'Content-Type': 'text/plain' });
6   res.end('Hello World!\n');
7 });
8
9 // starts a simple http server locally on port 3000
10 server.listen(3000, '127.0.0.1', () => {
11   console.log('Listening on 127.0.0.1:3000');
12 });
13
14 // run with `node server.mjs`
```

JavaScript

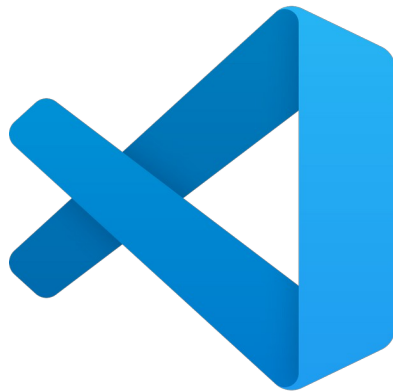
Copiar al portapapeles

Learn more what Node.js is able to offer with our [Learning materials](#).

<https://nodejs.org/>

Node.js

Editor de JavaScript

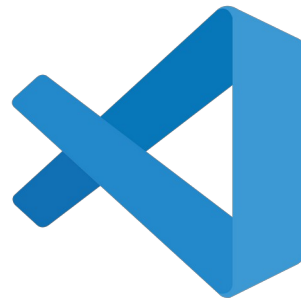


Visual Studio Code

<https://code.visualstudio.com/>

Editor de JavaScript

- Usaremos **Visual Studio Code** para desarrollar y depurar
- Usaremos algunas **APIs** que vienen incluidas con Node.js
- Usaremos **NPM** para instalar librerías externas



Ejercicio 1

Hello world!

ejer1-hello-world

- Crear una carpeta
- Crear un fichero **app.js** (no hay que hacer un html)

```
console.log("Hello World!");
```

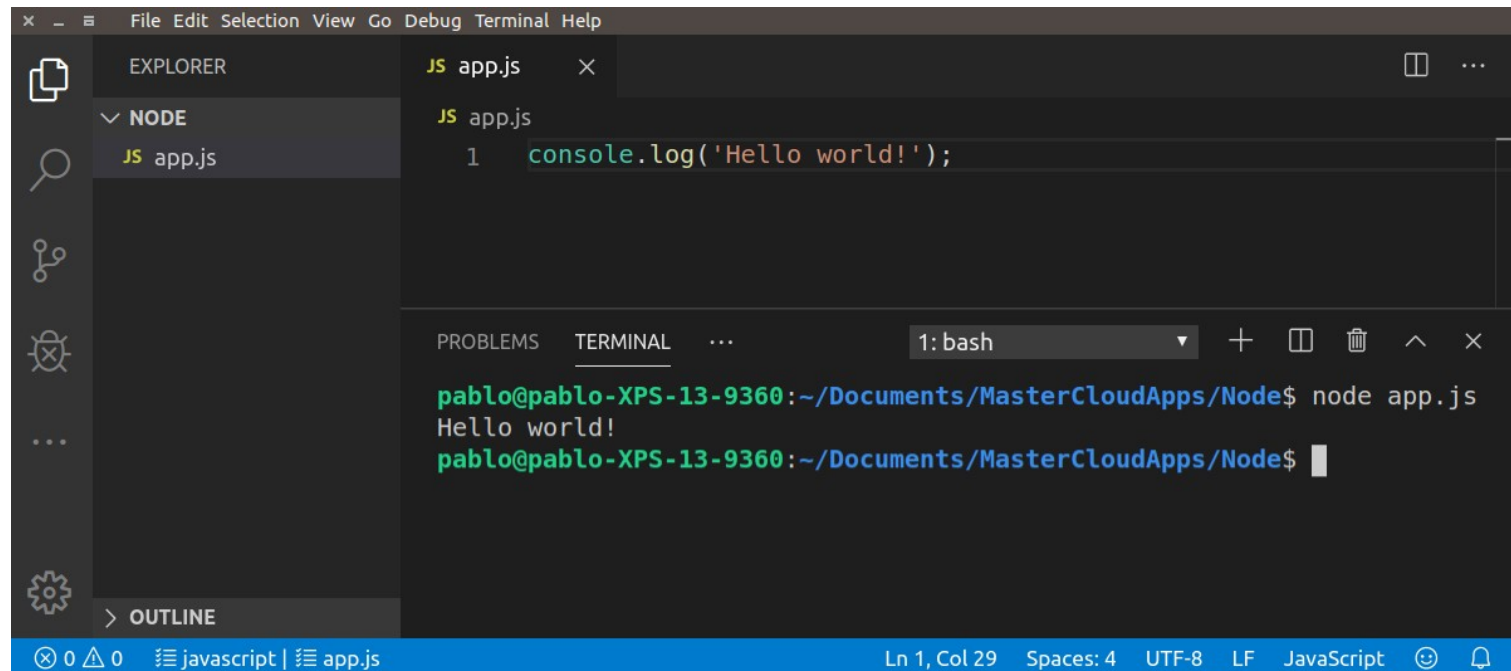
- Ejecutar el comando

```
$ node app.js
```


Visual Studio Code

- Abrimos la carpeta que tiene el código
- Ejecutamos comandos en el terminal integrado

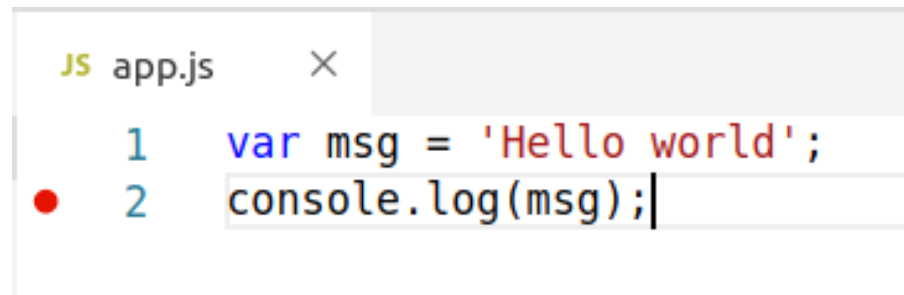
(Terminal > New Terminal)



The screenshot shows the Visual Studio Code interface with a dark theme. The Explorer sidebar on the left shows a folder named 'NODE' containing a file 'app.js'. The main editor area displays the content of 'app.js', which is a single line of JavaScript code: `console.log('Hello world!');`. Below the editor, the integrated terminal is open, showing a bash prompt. The user has entered the command `node app.js`, and the terminal has outputted `Hello world!`. The status bar at the bottom indicates the file is 'app.js' in the 'javascript' language, with 0 errors and 0 warnings. The terminal title bar shows '1: bash'.

Depuración en Visual Studio Code

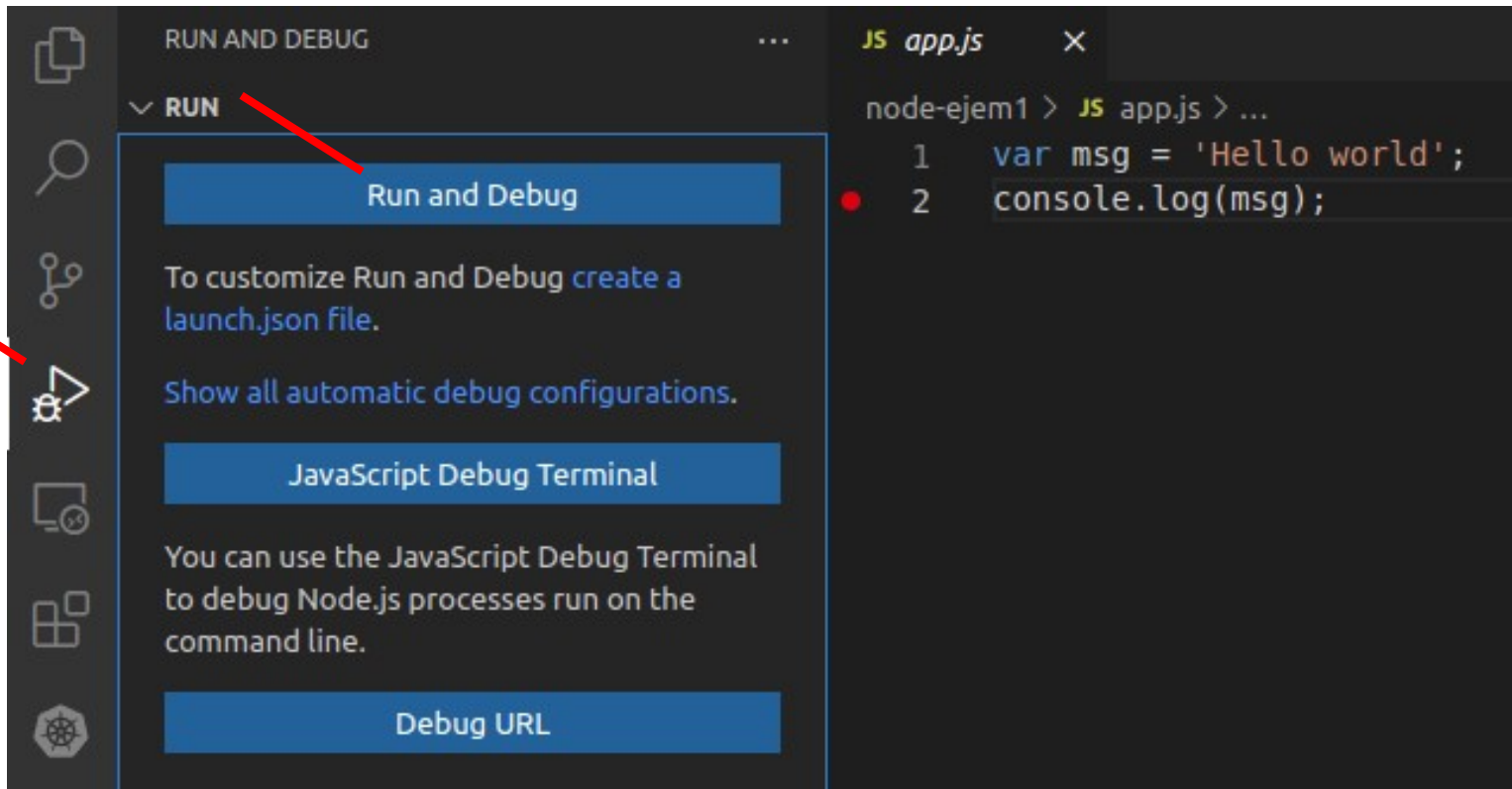
- Para poner un punto de ruptura se marca en la barra a la izquierda del código



```
JS app.js ×  
1 var msg = 'Hello world';  
● 2 console.log(msg);
```

<https://code.visualstudio.com/docs/nodejs/nodejs-tutorial>

Depuración en Visual Studio Code



Depuración en Visual Studio Code

